

Éléments de théorie pour la création des constructeurs. Le but de ce petit article est de donner la base de la théorie pour prouver la correction de l'algorithme qui construit les constructeurs pointés.

Les constructeurs font appel à des tuples imbriqués comme arguments, la première question est donc de savoir à quelles conditions un tuple peut-être considéré comme valide pour une règle. Il faudra ensuite montrer que des tuples pointés sont envoyés par l'algorithme vers des tuples non pointés. Enfin, il faut s'assurer que n tuples pointés de tailles n (en tant qu'objets d'une classe combinatoire) sont envoyés vers un seul tuple non pointé de taille n . Cela devrait suffire.

Quelques éléments de définition:

- On appelle classes auxiliaires les règles correspondant aux RuleName dans la bibliothèque UsainBoltz.
- On appelle constructeurs ce qu'on appelle builders au sein de la bibliothèque UsainBotlz.
- On note $|t|_z$ la taille d'un tuple t en tant qu'objet d'une classe combinatoire.

Quelques éléments de simplification par rapport aux vrais algorithmes point et unpoint:

- on considère qu'il existe une règle *Zero* qui correspond à la classe vide, cette classe est utilisée en lieu et place des simplifications qui ont place dans le vrai algorithme de pointage.
- on suppose que les constructeurs prennent toujours en argument des tuples et renvoient des tuples. Dans le cas de règles itérées qui renvoient en réalités de listes, on suppose que ces listes sont des tuples. Pour tout autre objet n'étant pas un tuple renvoyé (que ce soit par les constructeurs par défaut pour atoms, epsilons et markers, ou par les constructeurs fournis par l'utilisateur), on suppose que c'est un tuple contenant un seul élément.

On commence par définir l'algorithme de pointage d'une règle. Les règles sont supposées connues. Le code même si il ressemble à du Python est en réalité du pseudo code, du pseudo-python si vous voulez.

#On se permet les raccourcis suivant: $None-1 = None$; $(i-j) = \max(0, i-j)$. Très utile pour les contraintes de taille qui peuvent être non spécifiées et ne doivent pas passer sous zero.

```
def Point(A: Rule) -> Rule:
    switch type(A):
        case Atom():
            return Atom()
        case Epsilon() | Marker(-) | Zero():
            return Zero()
        case Union(A1, ..., An):
            return Union(Point(A1), ..., Point(An))
        case Product(A1, ..., An):
```

```

    return Union(
        Product(Point(A1), A2, ..., An),
        ...,
        Product(A1, ..., Point(An))
    case RuleName(name):
        return RuleName(name + "_P")
    case Set(A, leq = i, geq = j):
        return Product(A, Set(Point(A), leq = i-1, geq = j-1))
    case Cycle(A, leq = i, geq = j):
        return Product(A, Seq(Point(A), leq = i-1, geq = j-1))
    case Seq(A, leq = None, geq = None):
        return Union(Product(Seq(A, eq = 0), Point(A), Seq(A)))
    case Seq(A, leq = i, geq = j):
        #avec i != None.
        return Union(
            Product(Seq(A, eq = 0), Point(A), Seq(A, leq = i-1, geq = j-1)),
            Product(Seq(A, eq = 1), Point(A), Seq(A, leq = i-2, geq = j-2)),
            ...,
            Product(Seq(A, eq = i-1), Point(A), Seq(A, leq = 0, geq = j-i))
        case Seq(A, leq = None, geq = j):
            #avec j != None
            return Union(
                Product(Seq(A, eq = 0), Point(A), Seq(A, leq = None, geq = j-1)),
                ...,
                Product(Seq(A, eq = j-2), Point(A), Seq(A, leq = None, geq = 1)),
                Product(Seq(A, geq = j-1), Point(A), Seq(A)))

```

On définit maintenant l'algorithme de dépointage:

```

def Unpoint(A: Rule, t: tuple) -> tuple:
    switch type(A):
        case Union(A1, ..., An):
            (i, tu) = t
            return (i, Unpoint(Ai, tu))
        case Product(A1, ..., An):
            (i, (t1, ..., tn)) = t
            return (t1, ..., Unpoint(Ai, ti), ..., tn)
        case RuleName(-):
            return t
        case Cycle(A) | Set(A):
            (t1, (t2, ..., tn)) = t
            return (Unpoint(A, t1), t2, ..., tn)
        case Seq(A):
            (-, ((t_left_1, ..., t_left_n), t_pointed, (t_right_1, ..., t_right_m))) = t
            return (t_left_1, ..., t_left_n, Unpoint(A, t_mid), t_right_1, ..., t_right_m)

```

Pour, voici une définition plus formelle de $|t|_z$:

$$|t|_z =$$

Passons à la preuve,

Definition 1 (Tuple acceptable pour une règle.) *Un contexte Γ est la donnée d'une grammaire G et d'un dictionnaire qui associe à chaque classe auxiliaire une fonction qui prend en argument un tuple acceptable pour la classe auxiliaire et qui renvoie un nouvel objet. Pour Γ un contexte, A une règle de la grammaire*

de Γ , t un tuple, on dit que t est acceptable pour A et on note $G : A \vdash t$ si cette proposition peut être déduite des règles d'inférences que nous allons lister.

$$\frac{}{\Gamma : \text{Epsilon}() \vdash \text{Epsilon}()} \text{Epsilon}$$

$$\frac{}{\Gamma : \text{Atom}() \vdash \text{Atom}()} \text{Atom}$$

$$\frac{}{\Gamma : \text{Marker}(a') \vdash \text{Marker}(a')} \text{Marker}$$

$$\frac{i \in [1; n] \quad \Gamma : A_i \vdash t}{\Gamma : A_1 + \dots + A_n \vdash (i, t)} \text{Union } n \geq 2$$

$$\frac{\Gamma : A_1 \vdash t_1 \quad \dots \quad \Gamma : A_n \vdash t_n}{\Gamma : A_1 \times \dots \times A_n \vdash (t_1, \dots, t_n)} \text{Product } n \geq 2$$

$$\frac{\Gamma : A \vdash t_1 \quad \dots \quad \Gamma : A \vdash t_n}{\Gamma : \text{Seq}(A) \vdash (t_1, \dots, t_n)} \text{Sequence}$$

$$\frac{A \text{ est une classe auxiliaire associée à la règle } R \text{ et la fonction } B \text{ dans } G \quad \Gamma : R \vdash t}{\Gamma : A \vdash (t, B(t))} \text{Simplification}$$

La règle de simplification fait apparaître au dénominateur t et $B(t)$ au lieu de juste t pour obtenir certains théorèmes dans le cadre de cette théorie. En pratique, le t n'est pas nécessaire et on peut juste garder $B(t)$.

On définit aussi le pointage d'un contexte:

Definition 2 (Contexte pointé) Soit $\Gamma = \{G, A_1 : B_1, \dots, A_n : B_n\}$ un contexte. On définit le contexte pointé $\Gamma^\bullet$ par:

$$\Gamma^\bullet = \{G^\bullet, A_1 : B_1, \dots, A_n : B_n, A_1^\bullet : B_1^\bullet, \dots, A_n^\bullet : B_n^\bullet\}$$

où $B_i^\bullet(t) = B_i(\text{Unpoint}(t, R_i))$ avec $A_i : R_i \in G$.

Enfin, on donne une définition plus formelle de $|t|_z$:

$$\begin{aligned} |t|_z = & \\ |t = (t', B(t')) & \rightarrow |t'|_z \\ |t = (i, t') & \rightarrow |t'|_z \\ |t = (t_1, \dots, t_n) & \rightarrow |t_1|_z + \dots + |t_n|_z \\ |t = \text{Atom}() & \rightarrow 1 \\ |t = \text{Epsilon}() | t = \text{Marker}() & \rightarrow 0 \end{aligned}$$

On peut maintenant montrer les trois théorèmes suivants.

Theorem 1 (Unpoint réalise bien une opération de dépointage) Soit Γ, A, t un contexte, une règle et un tuple tel que A apparaisse dans G . $\Gamma^\bullet : A^\bullet \vdash t$ si et seulement si $\text{Unpoint}(t, A)$ est correctement défini et $\Gamma : A \vdash \text{Unpoint}(t, A)$.

Theorem 2 (Unpoint respecte la taille des tuples) Soit Γ, A, t tel que $\Gamma^\bullet : A^\bullet \vdash t$. Alors $|\text{Unpoint}(t, A)|_z = |t|_z$.

Theorem 3 (Il y a n tuples pointés pour un tuple non pointé de taille n) Soit Γ, A, t , tel que $\Gamma^\bullet : A^\bullet \vdash t$, alors $|\text{Unpoint}^{-1}(\{t\}, A)| = |t|_z$.

L'idée de la preuve est la même pour les trois, une induction sur A .
On commence par prouver un petit lemme utile:

Lemme 1 Soit Γ, A, t un contexte, une règle et un tuple. On a $\Gamma^\bullet : A^\bullet \vdash t$ avec A une règle apparaissant dans G si et seulement si $\Gamma : A \vdash t$.

Proof 1 Dans le sens direct, l'opération de pointage fait apparaître de nouvelles règles mais n'en supprime ni n'en modifie aucune. Donc A n'a pas été modifié, donc l'arbre de preuve utilisé pour obtenir que $\Gamma^\bullet : A^\bullet \vdash t$ permet de prouver $\Gamma : A \vdash t$.

On utilise le même argument dans l'autre sens.

Proof 2 (Unpoint réalise bien une opération de dépointage) Cas de bases:

- $A = \text{Epsilon}()$ donc $A^\bullet = \text{Zero}()$, on ne peut pas avoir $\text{Zero}() \vdash t$ car $\text{Zero}()$ n'est pas associé à une règle d'inférence. D'un autre côté, $\text{Unpoint}(t, \text{Epsilon}())$ lève une exception (donc n'est pas correctement défini).
- $\text{Marker}()$ et $\text{Zero}()$ sont très similaires à $\text{Epsilon}()$.
- $A = \text{Atom}()$ donc $A^\bullet = \text{Atom}()$. On a donc $\text{Unpoint}(t, A) = t$. Si $\Gamma^\bullet : \text{Atom}() \vdash t$, alors on peut utiliser la seule règle d'inférence faisant intervenir l'atome pour en déduire que $t = \text{Atom}()$. Donc, en appliquant cette même règle d'inférence on en déduit que $G : \text{Atom}() \vdash t$. Le sens inverse est identique.

Cas récurrents:

- $A = A_1 + \dots + A_n$, alors $A^\bullet = A_1^\bullet + \dots + A_n^\bullet$. On applique l'unique règle d'inférence associée à l'union, on en déduit que $t = (i, t')$ avec $1 \leq i \leq n$ et $\Gamma^\bullet : A_i^\bullet \vdash t'$. A apparaît dans G , donc A_i apparaît dans G , on peut donc utiliser notre induction. Par hypothèse d'induction, $\Gamma : A_i \vdash \text{Unpoint}(t', A_i)$ et donc, en appliquant la règle d'inférence pour l'union,

$$\Gamma : A \vdash (i, \text{Unpoint}(t', A_i)) = \text{Unpoint}((i, t'), A) = \text{Unpoint}(t, A)$$

. Le sens réciproque est identique.

- $A = A_1 \times \dots \times A_n$, alors $A^\bullet = A_1^\bullet \times A_2 \times \dots \times A_n + \dots + A_1 \times \dots \times A_{n-1} \times A_n^\bullet$. En appliquant la règle d'inférence pour l'union, on en déduit que $t = (i, t')$ avec $t' = (t_1, \dots, t_n)$. Puisque $\Gamma^\bullet : A_1 \times \dots \times A_i^\bullet \times \dots \times A_n \vdash (t_1, \dots, t_n)$, on peut cette fois ci appliquer la règle d'inférence pour le produit. Alors, pour $j \neq i$ $G^\bullet : A_j \vdash t_j$, or A_j apparait dans G car A apparait dans G , donc on peut appliquer notre lemme pour obtenir que $G : A_j \vdash t_j$. Par hypothèse d'induction, on sait aussi que $\Gamma : A_i \vdash \text{Unpoint}(t_i, A_i)$, on peut utiliser l'inférence pour le produit pour en déduire que

$$\Gamma : A_1 \times \dots \times A_n \vdash (t_1, \dots, \text{Unpoint}(t_i, A_i), \dots, t_n) = \text{Unpoint}(t, A)$$

Le sens réciproque est identique.

- Si A est une classe auxiliaire, la seule règle applicable est la simplification. Il existe donc un t et une règle R associée à A tel que $\Gamma^\bullet : A^\bullet \vdash B^\bullet(t)$ et $\Gamma^\bullet : R^\bullet \vdash t$. Par induction, $\Gamma : R \vdash \text{Unpoint}(t, R)$ et par simplification,

$$\Gamma : A \vdash B(\text{Unpoint}(t, R)) = B^\bullet(t) = \text{Unpoint}(B^\bullet(t), A)$$

Le sens inverse est identique.

Proof 3 (Unpoint respecte la taille des tuples) Cas de bases:

- $A = \text{Epsilon}()$, comme expliqué plus haut, ce cas ne peut pas se produire.
- Il en va de même pour $\text{Marker}()$ et $\text{Epsilon}()$.
- $A = \text{Atom}()$, alors $t = \text{Atom}()$, donc $t = \text{Unpoint}(t, A)$ et donc $|t|_z = \text{Unpoint}(t, A)_z$.

Cas récurifs:

- $A = A_1 + \dots + A_n$, donc $t = (i, t')$. Par induction, $|\text{Unpoint}(t', A_i)|_z = |t'|_z$. Or, $|(i, t')|_z = |t'|_z$ et $\text{Unpoint}((i, t'), A) = \text{Unpoint}(t', A_i)$, ce qui est suffisant pour conclure.
- A est une classe auxiliaire. On se donne t' , B et R tel que: $t = (t', B(t'))$ et $\Gamma^\bullet : R^\bullet \vdash t'$. À nouveau, on peut conclure en appliquant l'induction.